

房价预测模型验证报告

一、运行代码：可以顺利运行并复现

二、数据处理

1. 高度依赖硬编码，泛化能力弱

(1) 租期处理：硬编码匹配各种租期，如果原始租期中有如“5 年以上”等未见类别，无法正确转换，应尽量使用字符匹配规则取代字符串硬匹配

```
def parse_rent_term(s):
    # 解析 85 种租期格式为平均月数
    if pd.isnull(s): return np.nan
    s = str(s)
    # 替换年为月
    s = s.replace('1年以内', '0~12个月').replace('2年以内', '0~24个月')
    s = s.replace('3年以内', '0~36个月').replace('3年以上', '36~60个月') # 估算上限
    s = s.replace('1年以上', '12~24个月').replace('6个月以上', '6~12个月') # 估算
    s = s.replace('8个月以上', '8~12个月').replace('4个月以上', '4~12个月')
    s = s.replace('2个月以上', '2~12个月').replace('1个月以上', '1~12个月')
    s = s.replace('5个月以上', '5~12个月').replace('11个月以上', '11~12个月')
    s = s.replace('10个月以上', '10~12个月').replace('9个月以上', '9~12个月')
    s = s.replace('1年', '12个月').replace('2年', '24个月').replace('3年', '36个月')
    s = s.replace('1~2年', '12~24个月').replace('1~3年', '12~36个月')

    numbers = re.findall(r'(\d+)', s)
    if not numbers: return np.nan
    nums = [int(n) for n in numbers]

    # 根据关键字估算或计算平均值
    if '以上' in s: # (已替换大部分, 保留以防万一)
        return nums[0] + 6
    if '以内' in s: # (已替换大部分)
        return nums[0] / 2

    return np.mean(nums) # 处理 "X个月" 或 "X~Y个月"
```

(2) 分类标签编码：人工填写指定所有可能的分类标签，一方面很复杂，另一方面如果分类有变动修改困难。对于配套设施等一个样本属于多个类别的，可以使用“字符串拆分+展开汇总+获取唯一值”的方法自动获取所有的标签；对于一个样本属于一个类别的，可以直接使用独热编码

```

# 多标签二值化 (配套设施 - Point 5)
if '配套设施' in df.columns:
    amenities = ['洗衣机', '暖气', '空调', '床', '衣柜', '宽带', '电视', '热水器', '冰箱', '天然气']
    for amenity in amenities:
        df[f'配套_{amenity}'] = df['配套设施'].astype(str).str.contains(amenity, na=False).astype(int)

# 多标签二值化 (物业类别 - Point 4)
if '物业类别' in df.columns:
    main_types_wuye = ['别墅', '车库', '底商', '花园洋房', '普通住宅', '商业', '商业办公类', '商住两用', '写字楼', '公寓', '住宅', '单身公寓']
    for p_type in main_types_wuye:
        df[f'物业_{p_type}'] = df['物业类别'].astype(str).str.contains(p_type, na=False).astype(int)

# 多标签二值化 (建筑结构 - Point 2)
if '建筑结构' in df.columns:
    main_types_jiegou = ['塔楼', '板楼', '塔板结合', '平房']
    for j_type in main_types_jiegou:
        df[f'建筑_{j_type}'] = df['建筑结构'].astype(str).str.contains(j_type, na=False).astype(int)

# 多标签二值化 (产权描述 - Point 2)
if '产权描述' in df.columns:
    main_types_chaquan = ['房改房', '商品房', '经济适用房', '集资房', '使用权', '私产', '已购公房', '拆迁还建房', '二类经济适用房']
    for p_type in main_types_chaquan:
        df[f'产权_{p_type}'] = df['产权描述'].astype(str).str.contains(p_type, na=False).astype(int)

# --- 多标签二值化 供暖 ---
if '供暖' in df.columns:
    main_types_gongnuan = ['集中供暖', '自采暖', '无供暖', '平房']
    for j_type in main_types_gongnuan:
        df[f'供暖_{j_type}'] = df['供暖'].astype(str).str.contains(j_type, na=False).astype(int)

```

2. 数据提取错误

停车费里可能有复杂的文字表述，直接提取匹配到的第一个数作为停车费用会产生错误，应该使用更详尽的正则匹配方式

```

# 停车费
def get_first_number(s):
    if pd.isnull(s):
        return np.nan

    s = str(s)
    match = re.search(r'(\d+\.\d*)', s)

    if match:
        return float(match.group(0))
    else:
        return np.nan

df['停车费用'] = df['停车费用'].apply(get_first_number)

```

- ✓ 地下350元/月/位 加60管理费/月 (1384)
- ✓ 1000 (1145)
- ✓ 80 (845)
- ✓ 一元钱一小时，单次24小时内最高12元一次 (834)

三、样本选择：可能存在数据泄露问题

结合数据读取代码和数据拆分代码，推测数据的处理方式是：
comment_combined_rent.csv 文件中融合了 train 和 test 的所有数据，并新增了 train 列用于指示每个样本来自于 train 还是 test，所以 df 中含有全部的训练和测试数据。但是在填充板块、区县、城市缺失值的过程中，使用的是整个 df 的分组或全局众数，测试集数据暴露在数据处理步骤中，可能导致数据泄露问题

```

# --- STAGE 1: 加载并进行通用（非统计性）预处理 ---
print("--- STAGE 1: 加载并进行通用预处理... ---")
df = pd.read_csv('comment_combined_rent.csv', encoding='gbk')

#2. 删除不需要的列
df.drop(['环线位置', '年份', '开发商', '物业公司',
        '物业办公电话', 'coord_x', 'coord_y', '装修'
        ], axis=1, inplace=True)

# --- 1. 拆分数据 ---
print("--- 1. 正在拆分数据... ---")
train_df = df[df['train'] == 1].copy()
test_df = df[df['train'] == 0].copy()

# --- 1. L1 - 按层级填充（板块 < 区县 < 城市） ---
# （这是最高效的第一步）

# 1a. 填充 '板块'（使用其所在 '区县' 的众数）
print("--- L1: 正在按 '区县' 填充 '板块'... ---")
# （我们只在非空数据上计算规则）
sector_by_region_map = df[df['板块'].notnull()].groupby('区县')['板块'].apply(get_safe_mode).to_dict()
df['板块'] = df['板块'].fillna(df['区县'].map(sector_by_region_map))

# 1b. 填充 '区县'（使用其所在 '城市' 的众数）
print("--- L1: 正在按 '城市' 填充 '区县'... ---")
region_by_city_map = df[df['区县'].notnull()].groupby('城市')['区县'].apply(get_safe_mode).to_dict()
df['区县'] = df['区县'].fillna(df['城市'].map(region_by_city_map))

# --- 2. L2 - 按 K-Means 地理聚类填充（Fallback） ---
# 检查 L1 之后是否仍有缺失
missing_mask = df['板块'].isnull() | df['区县'].isnull()

# （根据您的保证，我们假设 Lon/Lat 在 missing_mask 中都存在）
if missing_mask.any():
    print(f"--- L2: 仍有 {missing_mask.sum()} 行缺失位置。正在启动 K-Means 地理填充... ---")

# 2a. 准备 K-Means 数据
# 'known_data' = 我们有完整地理信息（Lon, Lat, 板块, 区县）的行
known_data_mask = (
    df['lon'].notnull() & df['lat'].notnull() &
    df['板块'].notnull() & df['区县'].notnull()
)
known_data = df[known_data_mask].copy()

# 'target_data' = 我们有 Lon/Lat, 但 L1 填充失败的行
# （根据您的保证，missing_mask 的所有行都有 Lon/Lat）
target_data = df[missing_mask].copy()

```

四、代码编写规范

1. 代码冗余：代码修改过程中没有删除或注释掉原版代码，导致代码冗余

对于板块、区县、城市的缺失值填充，此前已经使用了分层众数填充和基于经纬度的 K-Means 算法填充，在之后又出现了用全局众数填充，对结果没有影响，但代码冗余

```

# --- 1. L1 - 按层级填充 (板块 < 区县 < 城市) ---
# (这是最高效的第一步)

# 1a. 填充 '板块' (使用其所在 '区县' 的众数)
print("--- L1: 正在按 '区县' 填充 '板块'... ---")
# (我们只在非空数据上计算规则)
sector_by_region_map = df[df['板块'].notnull()].groupby('区县')['板块'].apply(get_safe_mode).to_dict()
df['板块'] = df['板块'].fillna(df['区县'].map(sector_by_region_map))

# 1b. 填充 '区县' (使用其所在 '城市' 的众数)
print("--- L1: 正在按 '城市' 填充 '区县'... ---")
region_by_city_map = df[df['区县'].notnull()].groupby('城市')['区县'].apply(get_safe_mode).to_dict()
df['区县'] = df['区县'].fillna(df['城市'].map(region_by_city_map))

# --- 2. L2 - 按 K-Means 地理聚类填充 (Fallback) ---
# 检查 L1 之后是否仍有缺失
missing_mask = df['板块'].isnull() | df['区县'].isnull()

# (根据您的保证, 我们假设 Lon/Lat 在 missing_mask 中都存在)
if missing_mask.any():
    print(f"--- L2: 仍有 {missing_mask.sum()} 行缺失位置。正在启动 K-Means 地理填充... ---")

    # 2a. 准备 K-Means 数据
    # 'known_data' = 我们有完整地理信息 (Lon, Lat, 板块, 区县) 的行
    known_data_mask = (
        df['lon'].notnull() & df['lat'].notnull() &
        df['板块'].notnull() & df['区县'].notnull()
    )
    known_data = df[known_data_mask].copy()

    # 'target_data' = 我们有 Lon/Lat, 但 L1 填充失败的行
    # (根据您的保证, missing_mask 的所有行都有 Lon/Lat)
    target_data = df[missing_mask].copy()

# --- 2. 填充分组键 (板块/区县/城市) ---
print("--- 2. 正在填充分组键 (板块/区县/城市)... ---")
group_keys = ['板块', '区县', '城市']
rules_group_keys = {}
for col in group_keys:
    if col in train_df.columns:
        mode_val = train_df[col].mode()[0]
        rules_group_keys[col] = mode_val # 存储规则
        train_df[col] = train_df[col].fillna(mode_val)
        test_df[col] = test_df[col].fillna(mode_val)

```

2. **代码结构**: 数据处理过程中存在很多辅助函数, 但部分辅助函数定义于代码开头, 部分辅助函数定义于数据处理步骤内部。建议做好区分, 对于高度复用的辅助函数定义于开头, 对于某一数据列单独使用的辅助函数定义于数据处理步骤内部, 增强代码可读性和逻辑性

五、机器学习算法

模型的核心算法是线性回归+K-Means。其中 K-Means 算法是基于经纬度坐标聚类, 将所有房屋样本划分到 K 个地区上, 得到地区分类标签。但是, 原始数据中已经提供了城市、区县、板块的多层地区标签, 直接使用原始数据中的地区标签的效果是否会更好? 使用 K-Means 算法获得地区标签是否会引入因算法预测带来的新误差和干扰?

使用 K-Means 算法获得地区标签而非原始数据中的地区标签的可能原因是, 多层地区标签中没有合适的区分度 (城市、区县的区分度太低; 板块的区分度太高, 独热编码后特征过多且容易过拟合)。结合 K-Means 超参数调优的结果为 200, 城市数量约为 10, 区县数量约为 100, 板块数量约为 1000, 这种做法具有一定合理性。